AuthServlet.java

```
package com.harisudhan.harisudhanmart.controller;

import com.harisudhan.harisudhanmart.dto.UserResponseDTO;
import com.harisudhan.harisudhanmart.exception.AuthException;
import com.harisudhan.harisudhanmart.exception.ValidationException;
import com.harisudhan.harisudhanmart.model.User;
import com.harisudhan.harisudhanmart.service.AuthService;
import com.google.gson.JsonObject;
import java.io.BufferedReader;
import java.io.IOException;
import java.sql.SQLException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

/**
 * F1: registration and login. Handles /api/v1/auth/register, /api/v1/auth/login, /api/v1/auth/logout.
 * Session ID is regenerated on login (Section 2, engineering rule 3) to prevent session fixation.
 */
@WebServlet(urlPatterns = {"/api/v1/auth/register", "/api/v1/auth/login", "/api/v1/auth/logout"})
public class AuthServlet extends BaseServlet {

    private static final int SESSION_TIMEOUT_SECONDS = 1800;

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        String path = req.getServletPath();
        AuthService authService = new AuthService(daoFactory().userDAO());

        if (path.endsWith("/register")) {
            handleRegister(req, resp, authService);
        } else if (path.endsWith("/login")) {
            handleLogin(req, resp, authService);
        } else if (path.endsWith("/logout")) {
            handleLogout(req, resp);
        } else {
            writeError(resp, HttpServletResponse.SC_NOT_FOUND, "NOT_FOUND", "Unknown auth endpoint");
        }
    }

    private void handleRegister(HttpServletRequest req, HttpServletResponse resp, AuthService authService)
            throws IOException {
        JsonObject body = readJson(req);
        try {
            User user = authService.register(
                    getStr(body, "name"), getStr(body, "email"), getStr(body, "password"), getStr(body, "role"));
            writeJson(resp, HttpServletResponse.SC_CREATED, UserResponseDTO.fromEntity(user));
        } catch (ValidationException e) {
            writeError(resp, HttpServletResponse.SC_BAD_REQUEST, e.getFieldErrorCode(), e.getMessage());
        } catch (SQLException e) {
            writeError(resp, HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "DB_ERROR", "Registration failed");
        }
    }

    private void handleLogin(HttpServletRequest req, HttpServletResponse resp, AuthService authService)
            throws IOException {
        JsonObject body = readJson(req);
        try {
            User user = authService.login(getStr(body, "email"), getStr(body, "password"));

            // Regenerate session ID on login to prevent fixation.
```

```
            HttpSession oldSession = req.getSession(false);
            if (oldSession != null) {
                oldSession.invalidate();
            }
            HttpSession session = req.getSession(true);
            session.setMaxInactiveInterval(SESSION_TIMEOUT_SECONDS);
            session.setAttribute("userId", user.getId());
            session.setAttribute("role", user.getRole().name());
            session.setAttribute("name", user.getName());

            writeJson(resp, HttpServletResponse.SC_OK, UserResponseDTO.fromEntity(user));
        } catch (AuthException e) {
            writeError(resp, e.getStatusCode(), "AUTH_FAILED", e.getMessage());
        } catch (SQLException e) {
            writeError(resp, HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "DB_ERROR", "Login failed");
        }
    }

    private void handleLogout(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        HttpSession session = req.getSession(false);
        if (session != null) {
            session.invalidate();
        }
        writeJson(resp, HttpServletResponse.SC_OK, null);
    }

    private JsonObject readJson(HttpServletRequest req) throws IOException {
        StringBuilder sb = new StringBuilder();
        try (BufferedReader reader = req.getReader()) {
            String line;
            while ((line = reader.readLine()) != null) {
                sb.append(line);
            }
        }
        return com.google.gson.JsonParser.parseString(sb.length() == 0 ? "{}" : sb.toString()).getAsJsonObject();
    }

    private String getStr(JsonObject obj, String key) {
        return obj.has(key) && !obj.get(key).isJsonNull() ? obj.get(key).getAsString() : null;
    }
}
```